## Portfolio 5: Distributed and Cloud Database Architecture

**Project:** Stretto Technologies Database System

**Cloud Platform:** MongoDB Atlas

### 1. Distributed Cloud Architecture & Deployment

To transition Stretto Technologies from a localized development environment (PostgreSQL/pgAdmin) to a production-ready, globally accessible platform, the database was deployed on a managed cloud infrastructure using **MongoDB Atlas**.

Deploying to the cloud eliminates local hardware bottlenecks and provides automated infrastructure management. The system operates as a fully managed cloud cluster, enabling secure remote connections for distributed business administrators and procurement officers across Lagos and beyond, while ensuring enterprise-grade security and automated backups.

### 2. High Availability and Replication

To guarantee continuous uptime and protect against hardware failures, Stretto Technologies utilizes MongoDB Atlas's native **Replica Sets**.

- **Primary Node:** All active write operations—such as processing customer orders, updating inventory, and logging payments—are directed exclusively to the Primary node.
- **Secondary Nodes:** Data written to the Primary is asynchronously replicated across multiple Secondary nodes using the replication log (oplog).
- **Automated Failover:** If the Primary node experiences an unexpected outage or network drop, the cluster automatically triggers an election among the remaining nodes, promoting a Secondary node to Primary in a matter of seconds. This ensures zero data loss and uninterrupted availability for Stretto Technologies' e-commerce operations.

### 3. Horizontal Scaling via Sharding

As Stretto Technologies scales beyond its initial 50,000 transactional records into millions of entries, vertical scaling (upgrading server RAM or CPU) eventually hits a physical ceiling. To maintain sub-second query performance, the cloud database utilizes **Sharding** to horizontally partition data across multiple independent database servers (shards).

- **Shard Key Selection:** We designated customer_id as the **Shard Key** for the stretto_invoices collection.
- **Operational Impact:** This ensures that all historical purchase data and invoices belonging to a specific customer are grouped and routed to the exact same physical shard. When a customer checks their order history, the cluster queries only the relevant shard rather than scanning the entire global database, drastically reducing network overhead.

### 4. Consistency vs. Availability (CAP Theorem Trade-offs)

In a distributed cloud environment, the **CAP Theorem** dictates that a system can only provide two of the following three guarantees simultaneously: Consistency, Availability, and Partition Tolerance.

For Stretto Technologies, maintaining financial accuracy is paramount. Therefore, the cloud cluster is configured to prioritize **Consistency (CP)** over absolute availability during a network partition. If a network split occurs between the replica nodes, the database will temporarily reject write operations (such as new invoice placements) rather than risk processing a transaction against stale inventory or creating a split-brain payment discrepancy. Once network stability is restored, data synchronizes instantly across all nodes.

### 5. Deployment Evidence

The distributed cloud database cluster was successfully provisioned, deployed, and verified on the MongoDB Atlas platform.

